

DAY 6 · FastAPI learning notes

Dependencies: First Steps & Classes as Dependencies

Dependency injection · Depends · reusable Annotated aliases · sub-dependencies · class-based dependencies

Learning objectives

After working through these notes you should be able to:

- ✓ Explain what dependency injection is and which problems it solves.
- ✓ Write a dependency function and inject it with `Depends()`.
- ✓ Describe the order in which FastAPI resolves a dependency.
- ✓ Use a class as a dependency and explain why it gives better editor support than a dict.
- ✓ Tell apart the type annotation and the `Depends(...)` argument inside `Annotated`.

Note

This PDF keeps everything from [day6_summary_formatted.pdf](#) and adds a walkthrough of [day6/main.py](#). The folder also contains [day6_screen_recording.zip](#), a recorded teaching session for this day.

1. Dependencies - first steps

Dependency injection lets an endpoint declare something it needs, and FastAPI provides it automatically. Main uses: share logic or resources (e.g. database connections) across endpoints, handle security/authentication, and cut repeated code.

A dependency is usually just a function

day6/main.py

```
async def common_parameters(q: str | None = None, skip: int = 0, limit: int = 100):
    return {"q": q, "skip": skip, "limit": limit}

CommonsDep = Annotated[dict, Depends(common_parameters)]

@app.get("/items/")
async def read_items(common: CommonsDep):
    return commons

@app.get("/users/")
async def read_users(common: CommonsDep):
    return commons
```

A dependency function can take parameters exactly like a path operation function. FastAPI parses and validates them from the request the same way (query parameters, defaults, types).

`CommonsDep` is a reusable alias: two endpoints share the same three query parameters without repeating them – as your comment says, “that prevent redundant code”.

Resolution order

- 1 FastAPI sees `Depends(common_parameters)` on the parameter.
- 2 It inspects `common_parameters`' own parameters (`q`, `skip`, `limit`).

- 3 It extracts and validates those values from the incoming request.
- 4 It calls `common_parameters(...)` with those values.
- 5 The returned value is injected into the endpoint's `commons` parameter.
- 6 The endpoint function then runs normally.

Verified: `GET /items/?q=a&limit=2 → {"q": "a", "skip": 0, "limit": 2}`; `GET /users/ → {"q": null, "skip": 0, "limit": 100}`.

Common mistake

Pass the function, don't call it. `Depends(common_parameters)` is correct.

`Depends(common_parameters())` is wrong: it would call the function immediately instead of letting FastAPI call it for each request.

Sub-dependencies

Dependencies can depend on other dependencies, forming a tree. FastAPI resolves the whole chain automatically (e.g. `get_current_user` depends on `get_token`; an endpoint depends on `get_current_user`). This pattern is central to authentication later, and you use it with `yield` on Day 7.

From the Day 6 summary

```
def get_token():
    return "abc123"

def get_current_user(token: str = Depends(get_token)):
    return {"token": token}

@app.get("/profile")
def profile(user = Depends(get_current_user)):
    return user
```

Other notes

- Dependencies (and path operations) can be either `async def` or plain `def` – FastAPI handles both.
- Dependency parameters, validations and sub-dependencies all appear in the OpenAPI schema `/docs` automatically.

2. Classes as dependencies

Returning a plain dict works, but editors can't autocomplete dict keys because they don't know the shape. Classes fix this.

What makes something a dependency: being callable

A callable is anything Python can invoke with `()`. Functions are callable, but so are classes – `fluffy = Cat(name="Mr Fluffy")` is literally “calling” `Cat`. So a class can be a dependency: FastAPI inspects its `__init__` parameters exactly like a function's parameters (including sub-dependencies).

day6/main.py

```

class common_parameters_class:
    def __init__(self, q: str | None = None, skip: int = 0, limit: int = 100):
        self.q = q
        self.skip = skip
        self.limit = limit

item_filtered = Annotated[common_parameters_class, Depends(common_parameters_class)]

@app.get("/items_class/")
async def read_items(common: item_filtered):
    response = {}
    if common.q:
        response.update({"q": common.q})
    items = fake_items_db[common.skip:common.skip + common.limit]
    response.update({"items": items})
    return response

```

- 1 FastAPI calls `common_parameters_class(q=..., skip=..., limit=...)` and gets an **instance**.
- 2 That instance (not a dict) is passed as `commons`, so `commons.q`, `commons.skip`, `commons.limit` get autocomplete.
- 3 `fake_items_db[skip : skip + limit]` is list slicing – simple pagination.
- 4 `q` is added to the response only if it was sent.

Verified: `GET /items_class/?skip=1&limit=1&q=z` → `{"q": "z", "items": [{"item_name": "Bar"}]}`.

Type annotation vs. Depends() – two different jobs

In `Annotated[CommonQueryParams, Depends(CommonQueryParams)]` the class appears twice:

Part	Who uses it	Job
<code>Depends(CommonQueryParams)</code>	FastAPI	the real dependency: what FastAPI calls and extracts parameters from
first <code>CommonQueryParams</code>	your editor / type checker	says what type <code>commons</code> will be; no meaning to FastAPI's logic

You could even write `Annotated[Any, Depends(CommonQueryParams)]` and it would work identically at runtime – just without editor support.

The shortcut: Depends()

```
commons: Annotated[CommonQueryParams, Depends()]
```

When the dependency is a class FastAPI should instantiate, `Depends()` with no argument infers the class from the type annotation. It is purely a convenience – the docs say to ignore it if it seems more confusing than helpful; it changes nothing about behavior.

In your code

The second endpoint function is also named `read_items`. Each route is registered by its decorator before the name is reused, so both routes work – but unique function names are clearer.

Python convention: class names are CamelCase (`CommonQueryParams`), so `common_parameters_class` would usually be written that way. Behavior is the same.

3. Function vs class dependency

	Function returning a dict	Class
Injected value	<code>dict</code>	an instance of the class
Access	<code>commons["q"]</code>	<code>commons.q</code>
Editor autocomplete	no	yes
How FastAPI reads parameters	from the function's parameters	from <code>__init__</code> 's parameters

4. Common mistakes

- Calling the dependency inside `Depends(...)` instead of passing it.
- Thinking the first type in `Annotated[...]` is what FastAPI calls – it is the `Depends(...)` argument.
- Repeating the same parameters in many endpoints instead of making a dependency.

5. Revision summary

- Dependency injection: declare what you need with `Depends(...)`; FastAPI provides it per request.
- A dependency's parameters are read from the request exactly like an endpoint's.
- Reusable alias: `CommonsDep = Annotated[dict, Depends(common_parameters)]`.
- Classes are callables, so they can be dependencies; you get real attributes and autocomplete.
- Inside `Annotated`, the type is for the editor; `Depends(...)` is for FastAPI. `Depends()` can infer a class.

Source of the original summary: fastapi.tiangolo.com/tutorial/dependencies/ and [/tutorial/dependencies/classes-as-dependencies/](https://fastapi.tiangolo.com/tutorial/dependencies/classes-as-dependencies/)